Week-6

1. Example

Consider a regression problem in which the true relationship is given by:

$$g(x) = \sin 2\pi x, \;\; 0 \leqslant x \leqslant 1$$

The label is a noisy corrupted version of this function. The noise is an error term ϵ that is sampled from a zero mean Gaussian with variance of 0.1:

$$\epsilon \sim \mathcal{N}(0, 0.1)$$

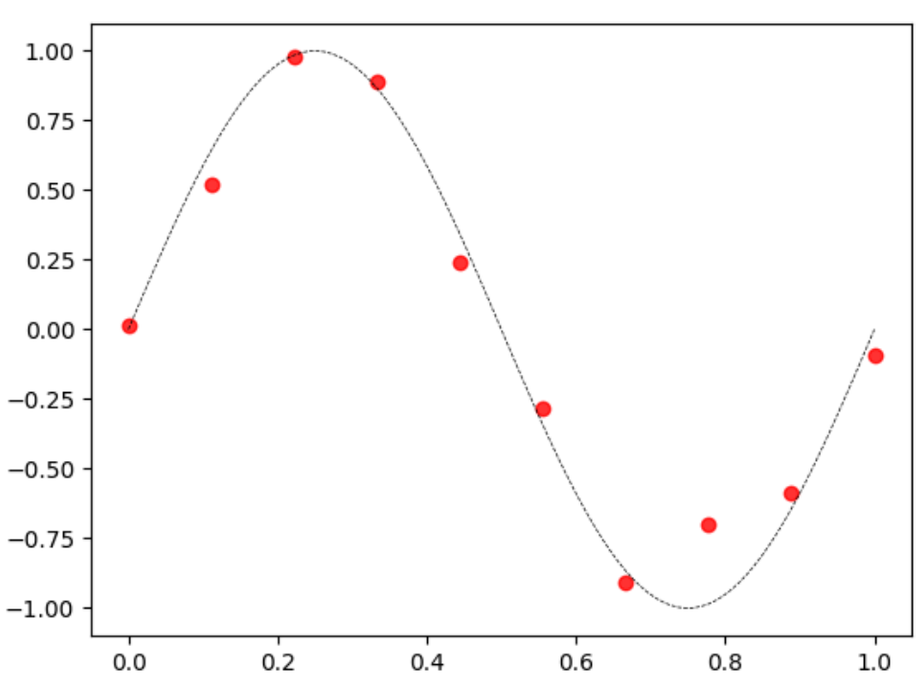
The label is therefore:

$$y_i \mid x_i \sim \mathcal{N}(g(x_i), 0.1)$$

This is the data-generation part. In reality, we don't know the functional form of $g(x)$. This is a toy dataset meant to aid in visualizing things.

Let us look at a dataset of size $n = 10$:

$$D = \{(x_1, y_1), \; \cdots, (x_{10}, y_{10})\}$$



We can turn this into a linear regression problem by forming non-linear polynomial features. Let us assume that we settle for a third degree polynomial. The feature vector is:

$$\mathbf{x} = \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix}$$

X is going to be a $d \times n$ matrix with $d = 4$ and $n = 10$. $d = 4$ since we are also adding an intercept. The label vector is $\mathbf{y}$. Concretely,

$$\mathbf{X} = \begin{bmatrix} 1 & \cdots & 1 \\ x_1 & \cdots & x_{10} \\ x_1^2 & \cdots & x_{10}^2 \\ x_1^3 & \cdots & x_{10}^3 \end{bmatrix}, \ \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_{10} \end{bmatrix}$$

The weight vector is going to be:

$$\mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \end{bmatrix}$$

Therefore, our model is:

$$\begin{aligned} f(x) &= w_0 + w_1 x + w_2 x^2 + w_3 x^3 \\ &= \mathbf{w}^T \mathbf{x} \end{aligned}$$

Remark: As far as the original problem is concerned, f is a non-linear function of the feature. After adding more (polynomial) features, f is linear in the weights. This is still called a linear regression problem as far the weights are concerned.

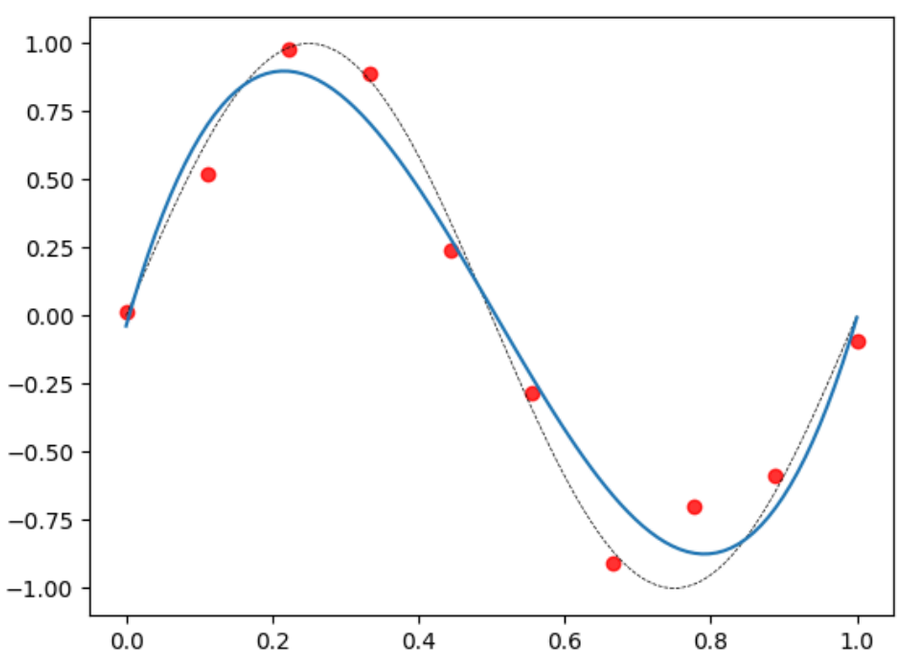
We can now learn the optimal weights by minimizing the SSE. We get:

$$\widehat{\mathbf{w}} = \left(\mathbf{X}\mathbf{X}^T\right)^{-1}\mathbf{X}\mathbf{y}$$

The weights we end up with are:

$$\widehat{\mathbf{w}} = \begin{bmatrix} -0.04 \\ 9.54 \\ -28.12 \\ 18.61 \end{bmatrix}$$

If we plot the function learnt, this is how it looks:

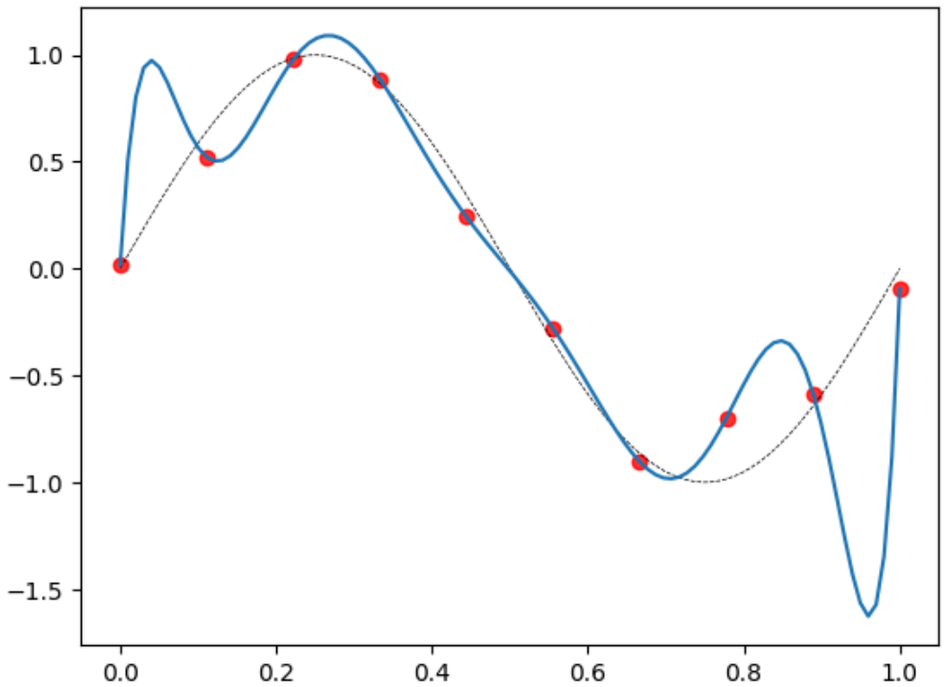


It seems like a reasonable fit.

2. Overfitting

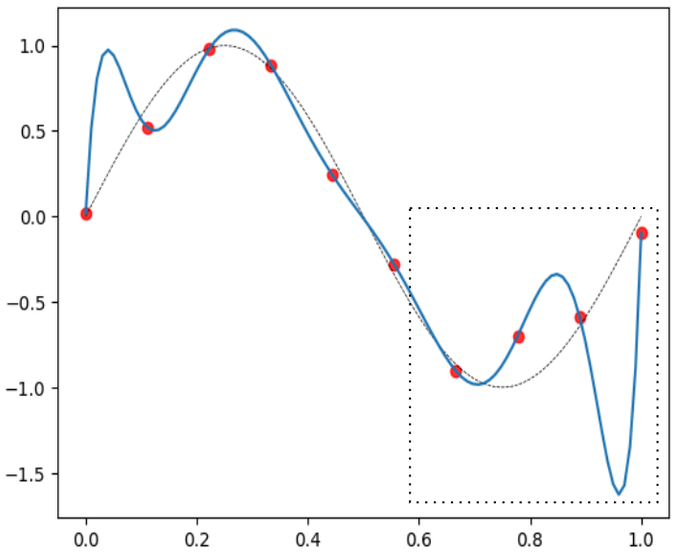
What if we change $p = 9$. The resulting data-matrix will be of shape $d \times n$ with $d = 10, n = 10$.

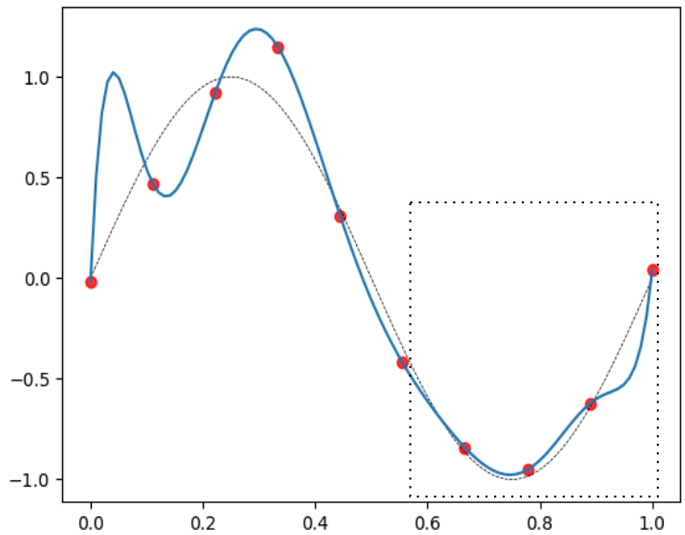
$$\begin{aligned} f(x) &= w_0 + w_1 x + \cdots + w_9 x^9 \\ &= \mathbf{w}^T \mathbf{x} \end{aligned}$$



$$\widehat{\mathbf{w}} = \begin{bmatrix} 0.02 \\ 62.12 \\ -1383.34 \\ 13019.05 \\ -63619.93 \\ 178687.61 \\ -300179.44 \\ 297927.91 \\ -160994.34 \\ 36480.26 \end{bmatrix}$$

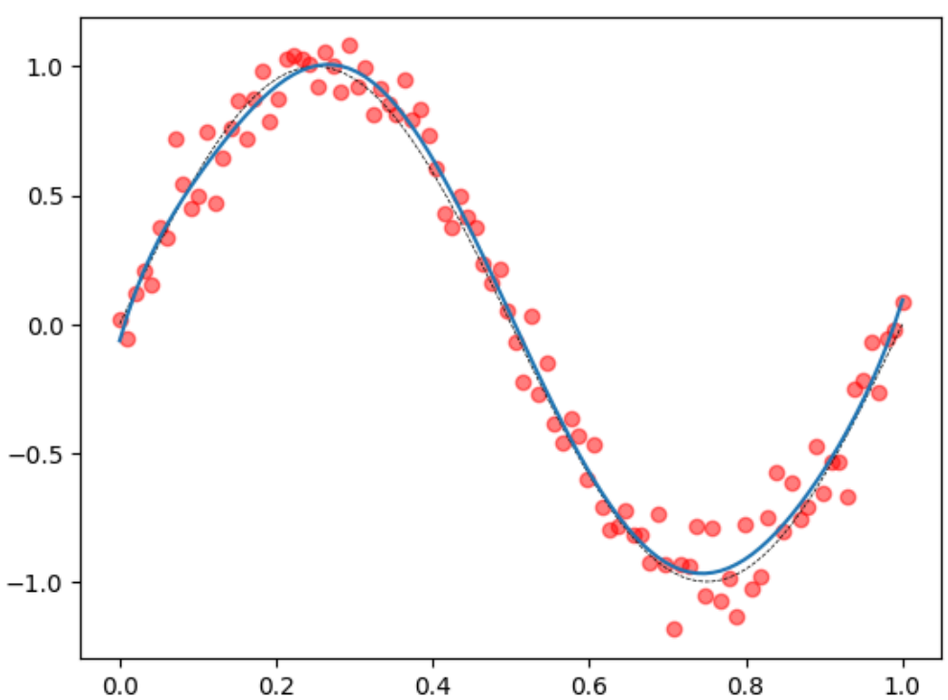
The model fits the noise in addition to the signal. This problem is termed **overfitting**. Also note the really high magnitudes for the components of the weight vector. An overfit model is said to be a **high variance model**: small changes in the dataset can result in wildly different models. For example:



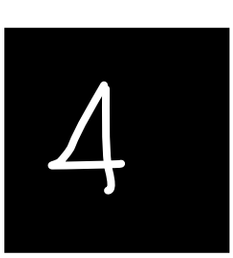


Note the boxed region. A small change in the position of one or two data-points caused a big change in the fitted curve.

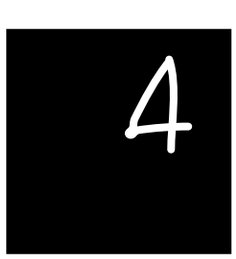
Increasing the data or augmenting it (data-augmentation) is one way of mitigating the problem of overfitting. For $n = 100$, the result is as follows:



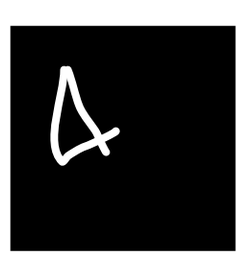
Data Augmentation is a popular technique for augmenting images



Translate



Rotation



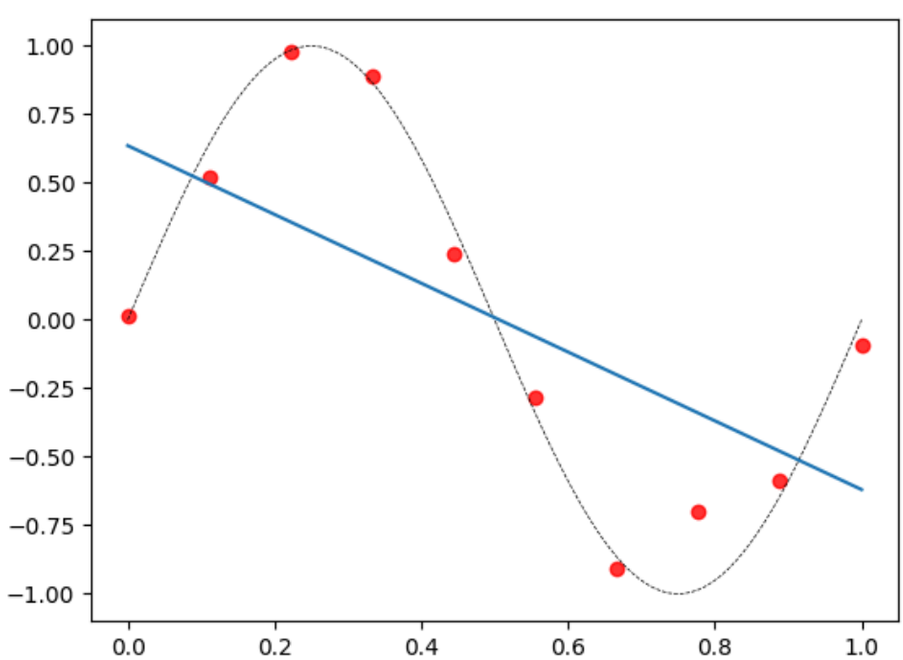
Coming back to the regression problem, the weight vector for this model is:

$$\widehat{\mathbf{w}} = \begin{bmatrix} -0.06 \\ 10.91 \\ -85.89 \\ 609.29 \\ -2557.32 \\ 5953.79 \\ -8253.17 \\ 6949.81 \\ -3318.79 \\ 691.52 \end{bmatrix}$$

Note the relatively smaller values for the weights. Models that are really complex tend to overfit. A high-degree polynomial is one such example.

3. Underfitting

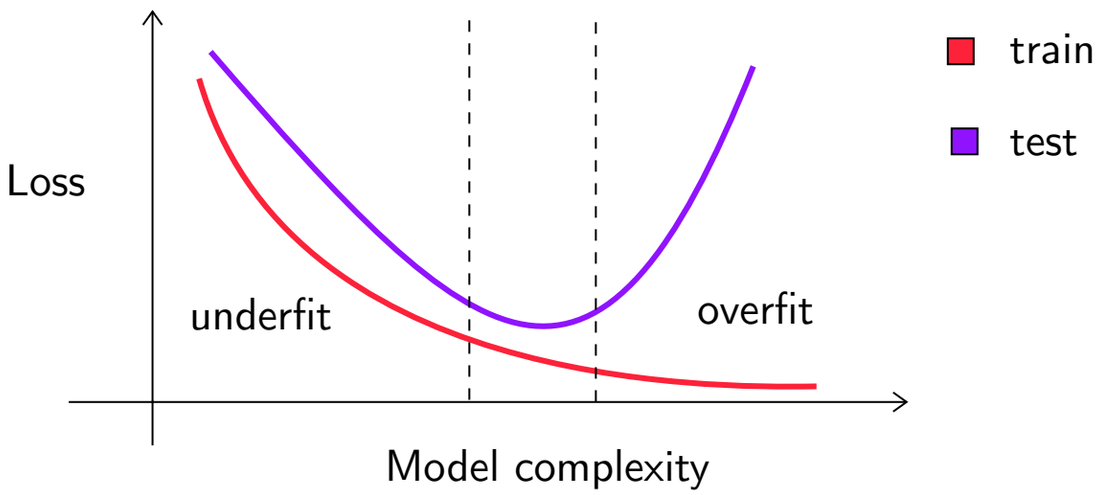
Underfitting is the opposite problem of being unable to capture the signal in the data:



A line is too simple for the given data and completely misses the patterns. When a model underfits, it is usually a good idea to fit a more complex model.

4. Model Complexity

If the train loss is low and the test loss is high, the model is overfitting. If the train loss is itself high, the model is underfitting. Here is a general plot of train and test loss with respect to model complexity:



Note how the training loss keeps decreasing as the models get more and more complex.

5. Regularization

Regularization is a method to mitigate overfitting. We can't always resort to adding more training data. In such situations, we impose an a constraint on the model's weights. We study two forms of regularization:

5.1. Ridge Regression

The loss function is modified as follows. This is called the regularized loss function:

$$L(\mathbf{w}) = \underbrace{\frac{1}{2} \cdot \sum_{i=1}^{n} \left(\mathbf{w}^T \mathbf{x}_i - y_i\right)^2}_{\text{SSE}} + \underbrace{\frac{\lambda}{2} ||\mathbf{w}||_2^2}_{\text{regularization term}} \tag{1}$$

where:

$$||\mathbf{w}||_2^2 = w_1^2 + \cdots + w_d^2$$

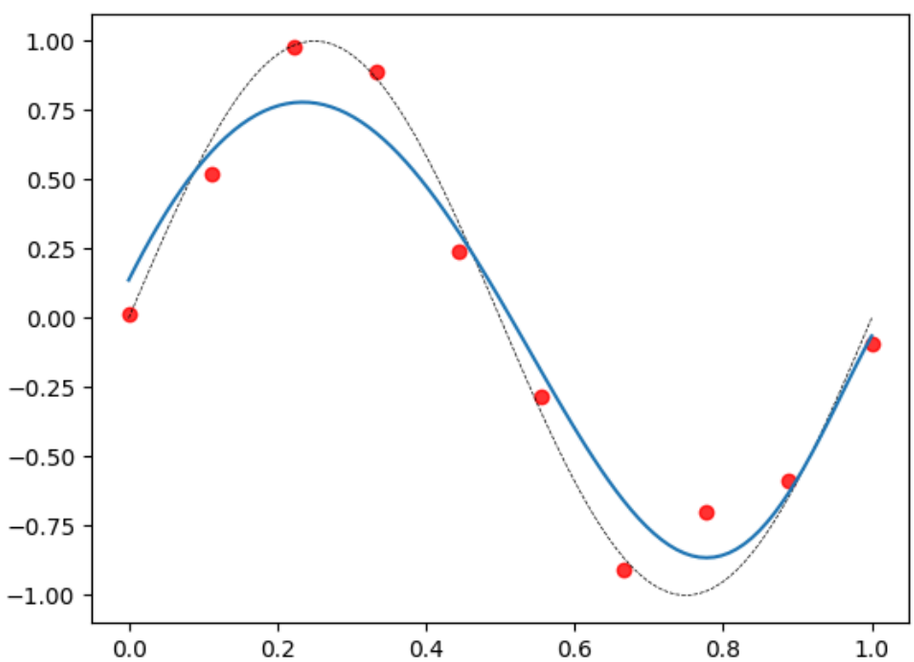
There are two aspects to the regularized loss:

- data-dependent loss: SSE
- model-dependent loss: regularization term
  - controls the complexity of the model

$||\mathbf{w}||_2$ is the L_2 norm of $\mathbf{w}$. This is the usual Euclidean norm. λ is called the regularization rate and is a **hyperparameter**. The difference between parameters and hyperparameters:

- parameters are learnt from data
- hyperparameters are chosen prior to the learning
  - however, the best hyperparameter for a given problem is still determined by making use of the (validation) data

For $\lambda = 0.001$, this is the resulting model:



The corresponding weight vector:

$$\widehat{\mathbf{w}} = \begin{bmatrix} 0.14 \\ 5.38 \\ -10.52 \\ -4.02 \\ 2.33 \\ 4.89 \\ 4.44 \\ 2.22 \\ -0.81 \\ -4.10 \end{bmatrix}$$

Ridge regression has the effect of pushing the weights nearer to zero though not exactly to zero. Let us see how to solve for $\mathbf{w}$:

$$L(\mathbf{w}) = \underbrace{\frac{1}{2} \cdot \sum_{i=1}^{n} \left(\mathbf{w}^T \mathbf{x}_i - y_i\right)^2}_{\text{SSE}} + \underbrace{\frac{\lambda}{2} ||\mathbf{w}||_2^2}_{\text{regularization term}}$$

On setting the gradient to zero we have

$$\mathbf{X}\mathbf{X}^T \mathbf{w} - \mathbf{X}\mathbf{y} + \lambda \mathbf{w} = 0$$

Simplifying:

$$\left(\mathbf{X}\mathbf{X}^T + \lambda \mathbf{I}\right) \mathbf{w} = \mathbf{X}\mathbf{y}$$

Therefore:

$$\widehat{\mathbf{w}} = \left(\mathbf{X}\mathbf{X}^T + \lambda \mathbf{I}\right)^{-1} \mathbf{X}\mathbf{y}$$

5.2. LASSO

$$L(\mathbf{w}) = \underbrace{\frac{1}{2} \cdot \sum_{i=1}^{n} \left(\mathbf{w}^T \mathbf{x}_i - y_i\right)^2}_{\text{SSE}} + \underbrace{\lambda ||\mathbf{w}||_1}_{\text{regularization term}}$$

where

$$||\mathbf{w}||_1 = |w_1| + \cdots + |w|_d$$

is the L_1 norm. It is also the Manhattan distance of the point $\mathbf{w}$ from the origin in $\mathbb{R}^d$. No closed form solution can be found for LASSO regression.

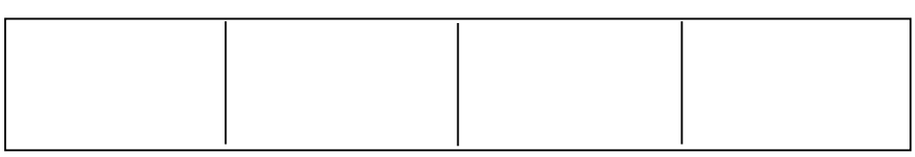
LASSO: Least Absolute Shrinkage Selection Operator

6. Cross Validation

As mentioned earlier, λ is called hyperparameter. It is chosen using a process called cross validation.

$k-$ Fold cross validation

$k = 4$ $\qquad$ $n = 100$



Search for λ in $\left[10^{-4}, 10^{-3}, \cdots, 10^{2}\right]$
For each of value of λ:

- Train on $k-1$ folds and test on the remaining fold.
- Do this k times and average the validation losses

Choose the λ that gives least validation loss. This process of determining the best value of λ for a given problem is called hyperparameter tuning.

7. Probabilistic view

- Prior

  - you start with small parameter values (hunch)
- Likelihood
  - how likely is it that we observe a dataset D given the parameters
- Posterior
  - how does the distribution over the parameters change after seeing
    the dataset

$$\text{Posterior} \propto \text{Prior} \times \text{Likelihood}$$

$$\mathbf{w} \sim \mathcal{N}\left(\mathbf{0}, \gamma^2 \mathbf{I}\right)$$

Multivariate Gaussian with scalar covariance $\rightarrow$ you can decompose it into a product of d univariate Gaussians.

$$P(\mathbf{w}|D) \propto P(\mathbf{w}) \times P(D|\mathbf{w})$$

$$P(\mathbf{w}) \propto \exp\left[\frac{-(w_1 - 0)^2}{2\gamma^2}\right] \cdots \exp\left[\frac{-(w_d - 0)^2}{2\gamma^2}\right]$$

$$P(\mathbf{w}) \propto \exp\left[\frac{-\sum_{i=1}^{d} w_i^2}{2\gamma^2}\right]$$

We know that:

$$y_i \sim \mathcal{N}\left(\mathbf{w}^T \mathbf{x}_i, \sigma^2\right)$$

Likelihood

$$P(D|\mathbf{w}) \propto \prod_{i=1}^{n} \exp\left[\frac{-\left(y_i - \mathbf{w}^T \mathbf{x}_i\right)^2}{2\sigma^2}\right]$$

Posterior

This is actually the log of the posterior after removing some terms which are not important for the optimization problem.

$$-\sum_{i=1}^{n} \frac{\left(y_i - \mathbf{w}^T \mathbf{x}_i\right)^2}{2\sigma^2} - \frac{\sum_{j=1}^{d} w_j^2}{2\gamma^2}$$

Maximizing the posterior $\rightarrow$ MAP

If we multiply the MAP objective by -1, we get the regularized loss for ridge regression (and the corresponding minimization problem).

$$L(\mathbf{w}) = \sum_{i=1}^{n} \frac{\left(y_i - \mathbf{w}^T \mathbf{x}_i\right)^2}{2\sigma^2} + \frac{1}{2\gamma^2}||\mathbf{w}||_2^2$$

- MAP estimate with a zero mean Gaussian prior is the same as the solution to the ridge regression problem.
- MAP estimate with a zero mean Laplacian prior is the same as the solution to the LASSO regression problem.

8. Geometric view

$$L(\mathbf{w}) = \underbrace{\frac{1}{2} \cdot \sum_{i=1}^{n} \left(\mathbf{w}^T \mathbf{x}_i - y_i\right)^2}_{\text{SSE}} + \underbrace{\frac{\lambda}{2}||\mathbf{w}||_2^2}_{\text{regularization term}} \tag{2}$$

$$\min_{\mathbf{w}} \; L(\mathbf{w})$$

Minimizing this is equivalent to the following constrained optimization problem:

$$\min_{\mathbf{w}} \quad \frac{1}{2} \cdot \sum_{i=1}^{n} \left(\mathbf{w}^T \mathbf{x}_i - y_i\right)^2$$

subject to

$$||\mathbf{w}||_2^2 \leqslant \theta$$

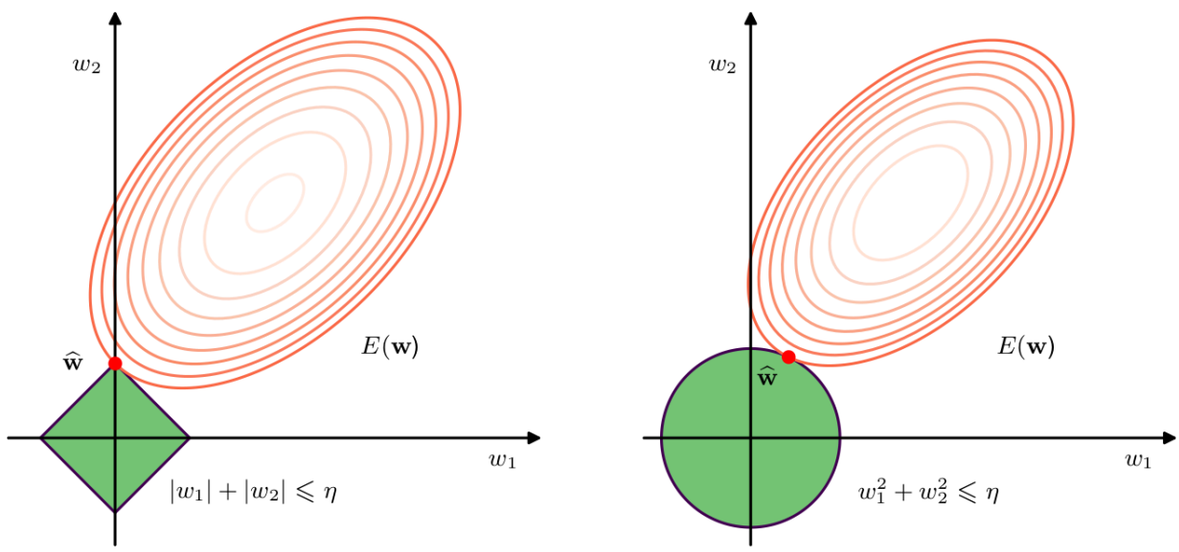


Figure 9.6 Plot of the contours of the unregularized error function (red) along with the constraint region (9.20) for the lasso regularizer $q = 1$ on the left, and the quadratic regularizer $q = 2$ on the right, in which the optimum value for the parameter vector $\mathbf{w}$ is denoted by $\widehat{\mathbf{w}}$. The lasso gives a sparse solution in which $\widehat{w}_1 = 0$, whereas the quadratic regularizer simply reduces w_1 to a smaller value.

Figure 1: Bishop, DL, Pg 266

8.1. Contours of the objective

Using $||\mathbf{v}||^2 = \mathbf{v}^T\mathbf{v}$ on both sides:

$$||\mathbf{X}^T\mathbf{w} - \mathbf{y}||^2 = ||\mathbf{X}^T\widehat{\mathbf{w}}_{ML} - \mathbf{y}||^2 + c$$

$$\left(\mathbf{X}^T\mathbf{w} - \mathbf{y}\right)^T\left(\mathbf{X}^T\mathbf{w} - \mathbf{y}\right) = \left(\mathbf{X}^T\widehat{\mathbf{w}}_{ML} - \mathbf{y}\right)^T\left(\mathbf{X}^T\widehat{\mathbf{w}}_{ML} - \mathbf{y}\right) + c$$

$$\mathbf{w}^T\mathbf{X}\mathbf{X}^T\mathbf{w} - 2\mathbf{w}^T\mathbf{X}\mathbf{y} + \mathbf{y}^T\mathbf{y} = \widehat{\mathbf{w}}_{ML}^T\mathbf{X}\mathbf{X}^T\widehat{\mathbf{w}}_{ML} - 2\widehat{\mathbf{w}}_{ML}^T\mathbf{X}\mathbf{y} + \mathbf{y}^T\mathbf{y} + c$$

From the normal equations, we have $\mathbf{X}\mathbf{X}^T\widehat{\mathbf{w}}_{ML} = \mathbf{X}\mathbf{y}$. Using this, we get:

$$\mathbf{w}^T\mathbf{X}\mathbf{X}^T\mathbf{w} - 2\mathbf{w}^T\mathbf{X}\mathbf{X}^T\widehat{\mathbf{w}}_{ML} + \widehat{\mathbf{w}}_{ML}^T\mathbf{X}\mathbf{X}^T\widehat{\mathbf{w}}_{ML} = c$$

The LHS simplifies to:

$$(\mathbf{w} - \widehat{\mathbf{w}}_{ML})^T \left(\mathbf{X}\mathbf{X}^T\right)(\mathbf{w} - \widehat{\mathbf{w}}_{ML}) = c$$